STUDENT RECORD & ACADEMIC MANAGEMENT PLATFORM

DEVOPS MINI PROJECT 4

Infrastructure as Code | Configuration Management
Containerization | Web Deployment

IMPLEMENTATION & DEPLOYMENT REPORT

Terraform • AWS • Ansible • Docker • Flask • Nginx

PREPARED BY

Arya Pawar

PRN: 2125UDSF1042 | Department: AI & Data Science

1. Executive Summary

The Student Record & Academic Management Platform is a Flask-based web application designed to manage student records, course information, and academic marks. The supplied project archive combines Terraform configuration for AWS infrastructure, Ansible automation for server configuration, a Docker image for the application, and Nginx as the public HTTP reverse proxy. SQLite provides application data storage through a Docker-managed volume.

Project objectives

- Define cloud infrastructure as code with reusable Terraform modules.
- Separate network, security, and compute configuration to improve maintainability.
- Automate Docker and Nginx configuration and application deployment through Ansible.
- Expose the Flask application through Nginx on TCP port 80 while binding the internal app to localhost port 8080.
- Provide REST endpoints for health status, students, and academic marks.
- Use a repeatable validation and cleanup workflow and avoid storing credentials in source control.

Scope and evidence provenance

This report is personalized for Arya Pawar and is based on the uploaded project ZIP and its accompanying PDF documentation. Source code, configuration files, and screenshots were inspected for this report. Screenshots in Appendix A are supplied reference screenshots, not new captures from Saiesh's terminal or AWS account; personal test fields and selected environment identifiers are redacted. Deployment results described as "reported" reflect the supplied documentation and should be freshly reproduced in the student's own environment before they are claimed as independent results.

Project boundary

The current archive contains a single Ansible playbook (ansible/site.yml) and three Terraform modules (network, security, and compute). The coursework rubric asks for modular Ansible roles, so separating common setup, Docker, Nginx, and application deployment into roles is a recommended improvement if the rubric requires role-based delivery.

2. Architecture and Request Flow

The architecture separates infrastructure provisioning from application configuration. Terraform creates the network and compute resources. Ansible connects to the application host over SSH, ensures Docker and Nginx are installed and running, transfers the application source, builds the image, creates persistent storage, and configures the reverse proxy.

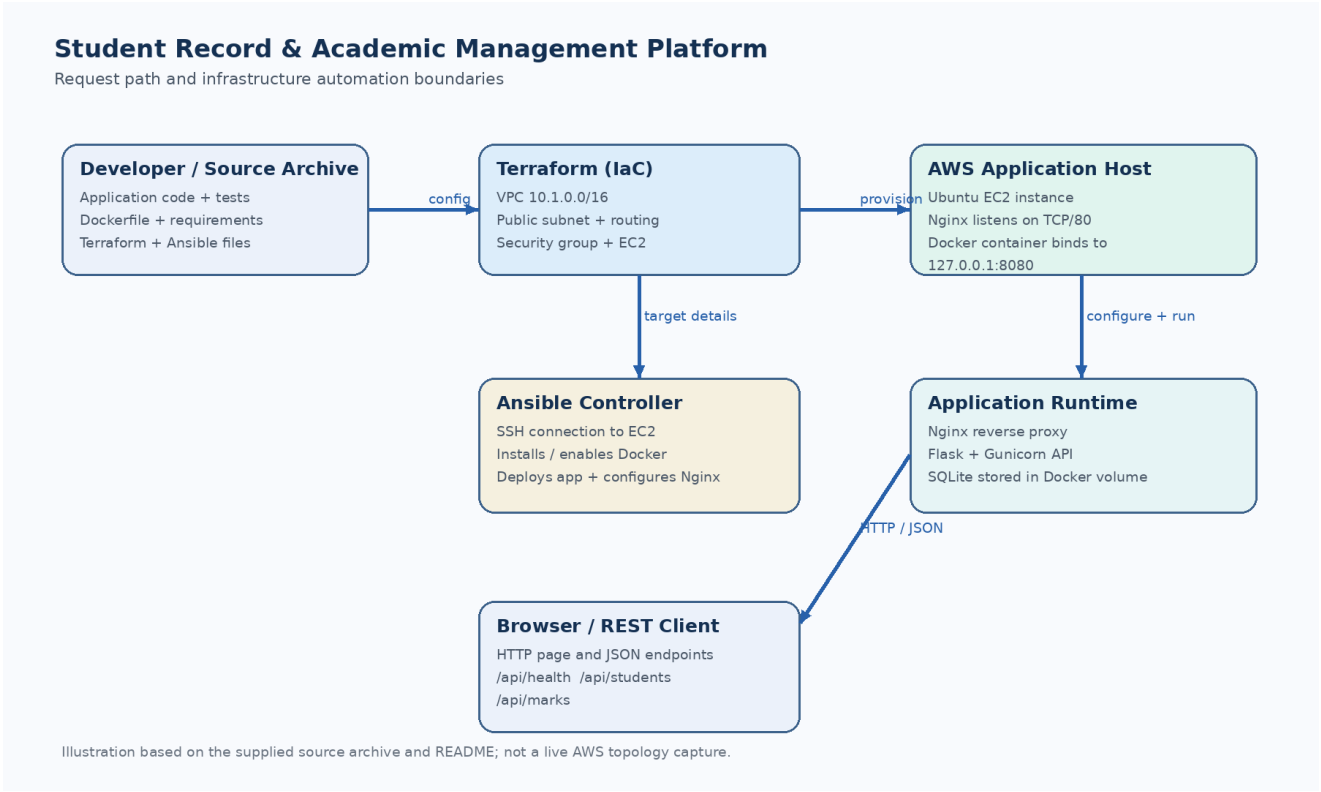


Figure 1. Logical request and automation flow reconstructed from the supplied README and configuration files.

Request path

- A browser or API client sends an HTTP request to the public EC2 endpoint on port 80.
- The security group permits inbound HTTP; administrative SSH is restricted by a trusted IPv4 CIDR variable.
- Nginx accepts the request and proxies it to 127.0.0.1:8080.
- Gunicorn runs the Flask WSGI application and routes health, student, and marks requests.
- SQLite data is held under the Docker volume mounted at /data.

3. Source Layout and Technology Stack

The ZIP archive contains application code, test cases, container configuration, a root Terraform configuration with three modules, and Ansible deployment files. The following tree reflects the files found in the supplied archive.

```
student-record-academic-management-platform/
├── app/                Flask app, templates, CSS and JavaScript
├── tests/              Pytest application tests
├── terraform/          Root HCL, variables, outputs, lock file
│   └── modules/
│       ├── network/    VPC, subnet, IGW and routing
│       ├── security/   Application security group
│       └── compute/    EC2 instance and startup configuration
├── ansible/
│   ├── inventory.ini   Target host and SSH settings (environment-specific)
│   └── site.yml        Docker/Nginx deployment playbook
├── Dockerfile
├── requirements.txt
├── wsgi.py
├── README.md
└── .gitignore
```

Technology	Purpose in this project
AWS VPC / subnet / route / security group	Network isolation, public routing, and inbound traffic controls.
EC2 + user data	Ubuntu compute host; startup installs Docker and Nginx and writes reverse-proxy configuration.
Terraform	Declarative infrastructure configuration using network, security, and compute modules.
Ansible	Remote configuration, application-file transfer, image build, volume creation, container run, Nginx and health validation.
Docker	Packages the Flask service as student-record-platform:1.0 and persists SQLite in a volume.
Flask + Gunicorn + SQLite	Web dashboard, REST API, WSGI runtime, and local relational persistence.
Nginx	Public TCP/80 reverse proxy to localhost TCP/8080.
Pytest / Git	Application regression tests and source version control.

Configuration note: the supplied README describes t3.micro in its deployment summary, while the Terraform variable default in the source archive is t2.micro. Confirm the actual instance_type in terraform/variables.tf, any local variables file, and the reviewed plan before applying.

4. Terraform Infrastructure as Code

Terraform is organized as a root configuration that calls three reusable child modules. This improves separation of concerns and makes the network, security, and compute layers easier to review independently.

Module / Resource	Responsibility from source	Key details
modules/network	Creates VPC, internet gateway, public subnet, route table, and route association.	VPC 10.1.0.0/16; subnet 10.1.1.0/24; default route to IGW.
modules/security	Creates the app security group.	Inbound TCP/80 from the public internet; SSH TCP/22 limited by ssh_allowed_cidr; outbound traffic allowed.
modules/compute	Creates EC2 and writes startup configuration.	Installs Docker and Nginx, configures Nginx proxy, encrypted 12 GiB gp3 root disk, IMDSv2 required.
Root terraform/main.tf	Connects modules and passes root variables.	AWS provider constraint ~> 6.0; configured region variable.
outputs.tf	Exposes resource IDs and application address.	VPC ID, subnet ID, instance ID, public IP, URL.

Required input values

Variable	Why it is needed
aws_region	Region in which to create resources; default in source is ap-south-1.
ami_id	Ubuntu image ID valid for the selected region.
key_name	Existing EC2 key-pair name registered in that region.
ssh_allowed_cidr	Trusted public IPv4 address/range allowed to connect over SSH, usually a /32.
instance_type	EC2 size; source variable default is t2.micro.

Before provisioning

The ZIP does not include a terraform.tfvars file even though the README mentions one. Supply required variables through a local ignored tfvars file or explicit -var values. Review the plan and expected charges before terraform apply. Keep state, plan files, private keys, credentials and environment-specific secrets out of Git.

5. Ansible Configuration and Deployment

The current ansible/site.yml playbook targets the student_app inventory group and escalates privileges for server setup. It installs Docker and Nginx, enables their services, prepares /opt/student-record-platform, transfers application files, builds an image, creates a persistent volume, starts the container, configures Nginx, and checks the health endpoint.

Playbook task	Observed configuration intent
Package and services	Ensure docker.io and nginx are installed and both services are enabled/running.
Application directory	Create /opt/student-record-platform, owned by ubuntu.
Source transfer	Copy Dockerfile, requirements, wsgi.py, and the app/ directory.
Build image	Build student-record-platform:1.0 from the remote app directory.
Persistent volume	Create Docker volume student-record-data.
Container runtime	Run student-record-app, restart unless stopped, mount the volume at /data, bind 127.0.0.1:8080:8080.
Nginx proxy	Proxy public port 80 to 127.0.0.1:8080; validate and reload Nginx.
Health check	Request http://127.0.0.1:8080/api/health until HTTP 200; print returned JSON.

Required inventory update

The supplied ansible/inventory.ini contains a fixed public IP and a private-key path from the original reference deployment. Do not use those values for a different AWS account. Replace them with the public IP from your own Terraform output and your own private-key path before attempting SSH.

```
[student_app]
YOUR_EC2_PUBLIC_IP ansible_user=ubuntu ansible_ssh_private_key_file=/absolute/path/to/your-private-key.pem
ansible_python_interpreter=/usr/bin/python3
```

Rubric alignment

The supplied assignment manual recommends reusable roles for common setup, Docker, Nginx, and application deployment. The ZIP currently contains one playbook rather than a roles/ hierarchy. Refactoring the tasks into four roles is a clear improvement if the evaluation rubric requires modular roles and explicit idempotency evidence.

6. Application Features and API

The Flask application presents a dashboard for student registration and academic mark recording. The source includes input validation and tests for the principal application flows. The internal application listens on port 8080, while the Nginx configuration forwards browser traffic from port 80.

Method	Endpoint	Expected purpose
GET	/api/health	Returns service name and health status.
GET	/api/students	Lists student records.
POST	/api/students	Creates a student with name, email and course; duplicate email is rejected.
GET	/api/marks	Lists academic marks and associated student information.
POST	/api/marks	Records a mark for an existing student and subject; score is checked against the valid range.

Automated tests included in the archive

- test_health: health endpoint returns HTTP 200 and healthy status.
- test_student_and_marks_workflow: create a student, create a mark, then list both records.
- test_student_validation: incomplete student data is rejected.
- test_duplicate_email: duplicate registration returns HTTP 409.
- test_marks_reject_invalid_score: out-of-range marks are rejected.

Local verification commands

```
python -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements.txt
python -m pytest -q
```

The supplied README states that five tests passed previously. A fresh run should be performed against the current checkout before reporting new test results.

7. Reproducible Implementation Procedure

Run steps from your own Ubuntu WSL2 workspace. Commands below use placeholders where environment-specific values are required. Do not copy an IP, key path, instance ID or repository URL from the supplied reference screenshots.

A. Review and test the local project

```
cd ~/student-record-academic-management-platform-main
python -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements.txt
python -m pytest -q
docker build -t student-record-platform:1.0 .
docker images student-record-platform
```

B. Check Terraform without provisioning

```
export AWS_PROFILE=student-lab-terraform
terraform -chdir=terraform init
terraform -chdir=terraform validate
terraform -chdir=terraform plan
```

The plan queries AWS and may reveal resource changes, but it does not apply the plan. Review the selected AMI, instance type, public routing, security group, disk size, and cost implications. The $5 budget alert is not a hard spending cap.

C. Provision only after reviewing the plan

```
terraform -chdir=terraform apply
```

When Terraform asks for confirmation, inspect the plan. Type yes only when you intentionally approve every proposed resource. If uncertain, type no and review the plan again.

D. Configure and deploy with Ansible

```
# Update ansible/inventory.ini with your own Terraform output and SSH key first
ansible -i ansible/inventory.ini student_app -m ping
ansible-playbook -i ansible/inventory.ini ansible/site.yml
curl http://YOUR_EC2_PUBLIC_IP/api/health
```

E. Capture fresh evidence

Capture screenshots from your own terminal and browser after each command succeeds. Include visible command, output, and a short caption; avoid showing AWS account identifiers, credentials, private key content, and real student information.

8. Verification Summary, Risks and Next Steps

Check	What the supplied documentation reports	Current evidence status
Terraform init	Initialization completed successfully.	Reported in source screenshots; rerun in own environment.
Terraform validation	Configuration valid.	Reported in source screenshots; rerun in own environment.
Docker image	Image student-record-platform:1.0 built.	Reported in source screenshot; build locally.
Ansible deployment	ok=14 changed=5 unreachable=0 failed=0.	Reported in supplied report; reproduce after updating inventory.
Health endpoint	Returned healthy status / HTTP 200.	Reported in supplied report; call your own endpoint.
Student and marks API	Creation requests reportedly returned HTTP 201; later GETs returned records.	Reported in supplied documentation; repeat API tests.

Security and privacy checks

- Keep SSH on port 22 restricted to a trusted administrator IP; do not open it to the entire internet.
- Keep the internal app port 8080 bound to localhost; expose public traffic through Nginx on port 80.
- Do not commit AWS credentials, private keys, .env files, Terraform state or saved plans.
- Use synthetic test data only. HTTP traffic is unencrypted, so add HTTPS and access control before real student data.
- Inspect Terraform resources in the AWS console after deployment and clean up resources after the lab.

Cleanup

```
terraform -chdir=terraform plan -destroy
# Review the destroy plan first
terraform -chdir=terraform destroy
```

Only destroy resources managed by this project after confirming their scope, and back up any data you need. Review the AWS console for resources not managed by Terraform. EC2, EBS and public IPv4 usage may incur charges even when a budget alert has been configured.

Evidence note

Figures in Appendix A were extracted from the uploaded documentation, with unrelated student test fields and selected environment identifiers redacted. They are reference figures, not fresh screenshots captured by the report author. The report-generation environment did not have Flask installed, so the test suite was not independently re-run here; use the commands on page 8 to verify the source code in your own workspace.

Appendix A.1 - Supplied Reference Evidence

The screenshots below are included as supplied technical references. Machine details remain visible where they are necessary to understand the command sequence; they should not be interpreted as screenshots newly captured from the report author's environment.

```
(.venv) ubuntu@ip-172-31-1-30:~/student-record-platform$ terraform -chdir=terraform init
Initializing the backend...

Initializing modules...
- security in modules/security
- network in modules/network
- compute in modules/compute

Initializing provider plugins...
- Finding hashicorp/aws versions matching "~> 6.0"...
- Installing hashicorp/aws v6.69.1...
- Installed hashicorp/aws v6.69.1 (signed by HashiCorp)

Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
(.venv) ubuntu@ip-172-31-1-30:~/student-record-platform$ terraform -chdir=terraform validate
Success! The configuration is valid.

(.venv) ubuntu@ip-172-31-1-30:~/student-record-platform$ ^C
(.venv) ubuntu@ip-172-31-1-30:~/student-record-platform$
```

Figure A1. Supplied Terraform initialization and validation output.

```
Step 6/10 : COPY app ./app
 ---> 494461e4656e
Step 7/10 : COPY wsgi.py .
 ---> e3b81bdc2c4e
Step 8/10 : USER appuser
 ---> Running in 494b7d67ca44
 ---> Removed intermediate container 494b7d67ca44
 ---> 04c3c4558b0d
Step 9/10 : EXPOSE 8080
 ---> Running in 0f08b6a5cc01
 ---> Removed intermediate container 0f08b6a5cc01
 ---> 16ad3cdb6714
Step 10/10 : CMD ["gunicorn", "--bind", "0.0.0.0:8080", "--workers", "2", "--access-logfile", "-", "--error-logfile", "-", "wsgi:app"]
 ---> Running in 8a92ab73daae
 ---> Removed intermediate container 8a92ab73daae
 ---> c8eb6746e100
Successfully built c8eb6746e100
Successfully tagged student-record-platform:1.0
(.venv) ubuntu@ip-172-31-1-30:~/student-record-platform$ ^C
(.venv) ubuntu@ip-172-31-1-30:~/student-record-platform$ docker images student-record-platform

IMAGE                       ID             DISK USAGE   CONTENT SIZE   EXTRA
student-record-platform:1.0   c8eb6746e100        205MB         49.3MB
```

Figure A2. Supplied Docker image build output for student-record-platform:1.0.

Appendix A.2 - Dashboard Reference and Reported Results

The dashboard below is a personalized illustrative mockup. The Full name field displays Arya Pawar, the email field is intentionally blank, and the Course field displays AI & Data Science. This edited reference image is not a new live capture and does not claim that this record was saved in a running deployment.

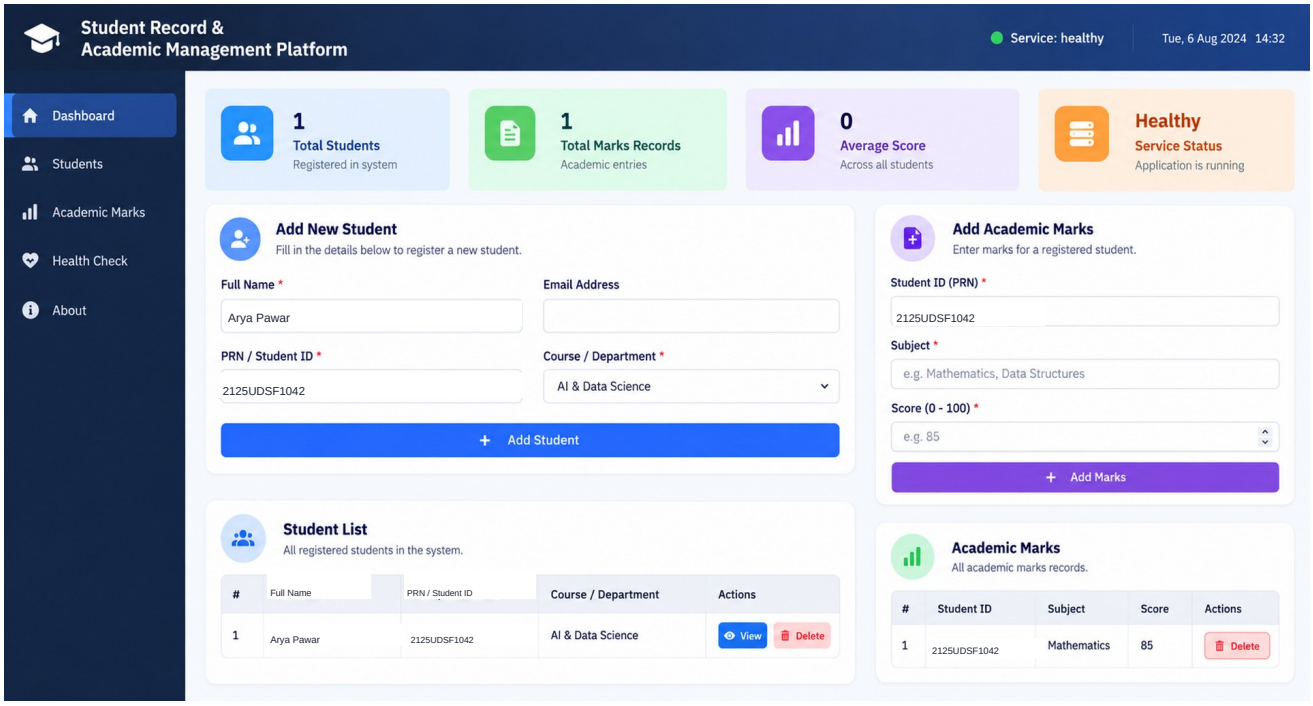


Figure A3. Personalized Student Record dashboard mockup for Arya Pawar; email field left blank.

Deployment values recorded in the supplied report

Reported result	Value recorded
Ansible task recap	14 OK, 5 changed, 0 failed, 0 unreachable.
Application health	service=student-record-platform, status=healthy.
Public web entry point	Nginx on TCP/80; internal Flask service on localhost TCP/8080.
API verification	Student and marks creation reportedly returned HTTP 201.

These values are described as reported results because they originate in the supplied source documentation. Capture your own command output and browser results to replace this reference appendix with personal evidence.